

Processor Design Project (PDP)

CESE4040 (Q4 2025-2026)

Course Description

Authors:

Georgi Gaydadjiev

Stefan Wong

Carlo Galuzzi

Mahmood Naderan-Tahan

Ignacio Garcia Ezquerro

Contents

Introduction 3

 Description 3

 Project structure..... 4

 Phase 1: Baseline bring-up and project planning (Weeks 1–2)..... 4

 Phase:2: Implementation and evaluation (Weeks 3–8) 5

 Learning objectives..... 5

 Assessment 6

 Expected prior knowledge 7

 Contact for students..... 7

Introduction

The Processor Design Project (PDP) is a hands-on, team-based integration course in which students take ownership of a complete hardware/software co-design problem on an embedded computing platform. Over the course of eight weeks, each team works with a baseline system consisting of a RISC-V soft core (RISCY), an LLVM-based cross-compiler toolchain, a Zynq-7000 FPGA development board, and a reference workload: a software implementation of the AES encryption algorithm.

Starting from this baseline, teams are expected to profile the system, identify its bottlenecks, and then design, implement, and evaluate improvements at both the architectural and the compiler level. The mandatory contributions for every group are:

- The implementation of selected instructions from the RISC-V scalar cryptography extension (aes32esi and aes32esmi), which accelerate the AES inner and final rounds.
- Applying the built-in Loop Unrolling optimization pass from LLVM toolchain for in the inner round.

On top of this obligatory part, each group formulates and justifies its own set of additional improvements, chosen to optimize a clearly defined target metric: execution latency, throughput, area, memory footprint, register pressure, resistance to side-channel attacks, or any other well-motivated objective.

The remainder of this document describes the course organization, the learning objectives, the assessment, and the contact information in more detail.

Description

This is an integration course designed to enable students to apply their knowledge of computer architecture and computer engineering to enhance the design of a general-purpose processor. The primary objective is to improve the performance of a RISC-V + LLVM-based computing platform, with a focus on both architectural and compiler-related aspects.

At the start of the course, students are provided with two resource packages:

1. A RISC-V System-on-Chip (SoC) implementation in Verilog, a reference AES encryption benchmark, and all supporting files required for simulation and FPGA deployment.
2. An archive containing the LLVM RISC-V cross-compiler toolchain.

To assess and validate their design improvements, each student group is equipped with a Zynq-7000 FPGA board, which serves as the target platform for implementation and benchmarking.

Project structure

The course is organized into two consecutive phases that run across the eight weeks of the quarter. The first two weeks are dedicated to bringing up the reference platform and producing an informed plan for the rest of the project. The remaining six weeks are dedicated to implementing, evaluating, and presenting the proposed improvements.

Phase 1: Baseline bring-up and project planning (Weeks 1–2)

The first two weeks are dedicated to getting the reference platform running end-to-end and to understanding its behaviour well enough to plan informed improvements for the rest of the project.

Task 1.1: Bring up the baseline system. At this stage, no modifications to the RISCY core or to the LLVM compiler are expected. Each group must:

- Synthesize the provided RISCY core together with its peripherals and supporting IPs using Vivado.
- Load the generated bitstream onto the FPGA board.
- Cross-compile the baseline AES software implementation with the provided LLVM toolchain.
- Run the compiled benchmark on the core executing on the FPGA.

A step-by-step manual covering each of these steps is provided on Brightspace.

Task 1.2: Analyze the baseline and plan Phase 2. Once the baseline is running, analyze the system to identify its bottlenecks. The choice of methodology is left to the group; analytical modelling, static analysis, dynamic profiling, or any combination thereof is acceptable, provided that the approach and its limitations are clearly stated. On the basis of this analysis, formulate an initial hypothesis describing how the system can be improved. Possible optimization targets include, but are not limited to:

- Lowest execution latency
- Highest throughput
- Minimal area utilization
- Minimal memory footprint or register pressure
- Security properties, e.g. resistance to side-channel attacks
- AES-specific LLVM optimization passes targeting a clearly stated goal

The main deliverable of Phase 1 is a detailed project plan for Phase 2 (Weeks 3–8). The plan must include internal tasks with explicit assignments to team members, their interdependencies, and built-in milestones to monitor progress. It must also identify the

state of the art the group will compare against in the background section of the intermediate report and describe the experimental methodology that will be used to obtain the quantitative results supporting the group's claims.

The intermediate report is made available on Brightspace as a quiz module which starts on 4/5/2026 and ends on 8/5/2026 at 23:59. One submission per group is expected where all group members consult with each other and answer the associated questions within that window.

Phase:2: Implementation and evaluation (Weeks 3–8)

In the second phase, groups implement the mandatory part (AES instruction plus the Loop Unrolling pass) and the improvements planned in Phase 1, evaluate the resulting system, and prepare the final deliverables. The project has a hard deadline at the end of Week 8, and students are expected to operate as a team throughout; the workload is substantial and requires careful division of tasks and continuous coordination.

Task 2.1: Implement the mandatory AES instructions. Implement and integrate the RISC-V scalar cryptography extension instructions `aes32esi` (final round) and `aes32esmi` (inner round) into the RISCY core, and expose them through the LLVM toolchain. Once these are functional, apply the LLVM built-in loop-unrolling pass to the AES inner-round instruction (`aes32esmi`) and assess its effect on the target metrics.

Task 2.2: Implement and evaluate the group-specific plan. Execute the plan produced during Phase 1: implement the proposed improvements, measure their impact using the methodology described in the intermediate report, and iterate as needed.

Task 2.3: Prepare for the final assessment. The assessment consists of:

- A presentation of 20 minutes, in which every group member must present part of the work;
- A 10-minute live demonstration of the final system;
- A 20-minute Q&A session.

Prior to the assessment session, groups must submit two items on Brightspace: the final presentation slides, and a complete archive of the project artefacts, including the modified RISCY RTL sources, the LLVM toolchain modifications, build and run scripts, and any other source files that have been written or changed during the project. More details regarding the submission deadline will be announced accordingly. All submissions are checked for originality.

Learning objectives

The course study goals are as follows:

1. Analyze processor architecture constraints and design requirements to evaluate the design space and select appropriate algorithms for implementing basic functional units.
2. Integrate knowledge from computer engineering, compilers, and computer architecture to design and optimize processor architectures.
3. Evaluate processor performance using standard benchmarks and formulate architectural improvements to enhance design metrics such as speed, area, and power consumption.
4. Design arithmetic and functional units and implement complete processors using FPGA technology by applying digital design principles.
5. Collaborate effectively within a small team to plan, coordinate, and complete a processor design project.
6. Communicate technical proposals, experimental results, and conclusions clearly and effectively in English, both in written reports and oral presentations, using appropriate technical terminology.

Assessment

To successfully complete the course, students are required to:

1. Complete two parts of a guided laboratory assignment.
2. Submit an intermediate report outlining their intended approach and planning.
3. Submit the final presentation slides before the assessment session;
4. Submit a complete archive of the project artefacts: RTL sources, LLVM modifications, build and run scripts, and any other source files written or modified during the project; before the assessment session, for originality checks;
5. Present their work in the final assessment session, including a live demo and a Q&A.

The final individual grade is determined based on:

- A group grade, derived from the quality of the intermediate report, the final presentation and demo, and the submitted project artifacts, and
- A peer review process within each group, assessing individual contributions.

Throughout the quarter, students are expected to provide regular progress updates. These updates are used to offer high-level guidance and ensure students stay on track toward achieving their design objectives.

If the final result is insufficient, repair options may be available in accordance with Article 17A (Times and number of examinations, paragraph 1) of the Teaching and Examination Regulations (TER).

Disclaimer: Course requirements and procedures are subject to change due to unforeseen circumstances or institutional decisions (see TER, Article 2, paragraph 5)

Expected prior knowledge

To successfully complete the project, students must possess a solid understanding of Digital Systems and Verilog. In addition, they are required to have successfully completed the following prerequisite courses: CESE4085 Modern Computer Architecture and CESE4130 Computer Engineering.

Contact for students

Name	Email	Office
Carlo Galuzzi	C.Galuzzi-2@tudelft.nl	36.HB 10.040
Georgi Gaydadjiev	G.N.Gaydadjiev@tudelft.nl	36.HB 10.030
Stefan Wong	J.S.S.M.Wong@ewi.tudelft.nl	36.HB 10.050
Mahmood Naderan	M.Naderan@tudelft.nl	36.HB 10.230
Ignacio García Ezquerro	I.GarciaEzquerro@tudelft.nl	36.HB 10.230
Pantazis Anagnostou	P.A.Anagnostou@tudelft.nl	36.HB 09.310
Ismail Bourhaeil	I.Bourhaeil@tudelft.nl	36.HB 10.250
Konstantinos Stavrakakis	K.Stavrakakis@tudelft.nl	36.HB 10.250
Emmanouil Arapidis	E.Arapidis@tudelft.nl	
Fouwad Mir	F.J.Mir@tudelft.nl	

The communication channels for this course are listed below:

- Brightspace: This is the primary and official course [web page](#). All announcements, assignments, etc, are made via this page.
- Teams: This is a supplementary communication channel used for per group or general technical messages. The name of each channel corresponds to the group number, and all students in the group plus course management team are added to that private channel. This way, the course team can track students progress and communicate in a more efficient manner.